

Group Members: Raffay Khan

Git: [RaffayKhan2502/InfoSec_Proj](#)

1. Introduction to Post-Quantum Cryptography (PQC)

Post-Quantum Cryptography (PQC) refers to cryptographic algorithms designed to be secure against the potential future threat of quantum computers. Quantum computers, once fully developed, will be able to break traditional cryptographic systems that rely on problems like integer factorization (RSA) and discrete logarithms (Elliptic Curve Cryptography). As a response to this emerging threat, PQC aims to provide cryptographic schemes that are resistant to the power of quantum algorithms like Shor's Algorithm, which efficiently solves problems that classical computers cannot.

Given the advancements in quantum computing, it is imperative to transition to new cryptographic standards that will safeguard our digital world. PQC algorithms are designed with quantum resistance in mind, ensuring secure communication even in a post-quantum era.

2. PQC Algorithm Used: X25519 (Elliptic Curve Diffie-Hellman)

For the implementation of the PQC algorithm in this web application, we used **X25519**, which is an elliptic curve Diffie-Hellman (ECDH) key exchange protocol. While not technically a post-quantum algorithm in the strict sense, X25519 is considered quantum-resistant against some quantum algorithms due to its reliance on the difficulty of the Elliptic Curve Discrete Logarithm Problem (ECDLP), which, while potentially solvable by quantum computers, is still resistant to certain quantum attacks.

X25519 is widely used due to its strong security properties, efficiency, and ease of implementation. It's suitable for this demonstration in a PQC-based cryptographic web application for key generation, encryption, and decryption.

3. Working Explanation

The web application implements three main cryptographic operations:

1. **Key Generation**
2. **Encryption**
3. **Decryption**

Here is a breakdown of the workings of these operations:

3.1 Key Generation

Key generation in this implementation uses **Elliptic Curve Diffie-Hellman (ECDH)** based on the X25519 curve. This algorithm allows two parties to securely exchange keys over an insecure communication channel. The steps are as follows:

- **Private Key Generation:** A private key is randomly generated using a secure cryptographic random number generator. This key is kept secret by the owner and should never be shared.
- **Public Key Derivation:** The public key is derived from the private key using the elliptic curve formula. The public key is shared with others, allowing them to encrypt messages or verify signatures made with the corresponding private key.

In the application, we generate the private and public keys using the **X25519** algorithm and provide them to the user. The public key is then shared with others to enable message encryption.

3.2 Encryption

The encryption process allows a user to encrypt a message using the recipient's public key. This ensures that only the intended recipient, who has the corresponding private key, can decrypt the message. The encryption steps are as follows:

- **Ephemeral Key Generation:** For added security, an ephemeral (temporary) public/private key pair is generated for each encryption operation. This ensures that even if a key is compromised, previous messages remain secure.
- **Shared Key Generation:** Using the recipient's public key and the sender's ephemeral private key, a **shared key** is derived through the **Diffie-Hellman** key exchange protocol. This shared key acts as a secret key used to encrypt the actual message.
- **Message Encryption:** The message is combined with the shared key, and the result is encoded using **Base64** to produce a safe, readable encrypted message. The encrypted message and the ephemeral public key are sent to the recipient.

3.3 Decryption

Decryption allows the recipient to retrieve the original message using their private key and the sender's ephemeral public key. The decryption process works as follows:

- **Private Key Usage:** The recipient uses their private key to perform another **Diffie-Hellman** key exchange with the ephemeral public key sent by the sender. This yields the same shared key that was used during encryption.
 - **Message Decryption:** Using the derived shared key, the encrypted message is decoded and the original message is retrieved.
 - **Base64 Decoding:** The encrypted message is first **Base64** decoded, and then the message part (without the shared key) is extracted and returned to the recipient.
-

4. Security Considerations

To enhance the security of the system, several measures were implemented:

- **Secure Key Generation:** The keys are generated securely using strong cryptographic algorithms (X25519), and private keys are never exposed.
- **Ephemeral Key Pairs:** Every encryption operation generates a new ephemeral key pair, ensuring forward secrecy—if an attacker intercepts a message, they cannot use the ephemeral key to decrypt other messages.
- **Base64 Encoding:** All sensitive data, such as encrypted messages and keys, is encoded using Base64 to ensure they are safely transmitted as text.

Additionally, standard web security practices were followed, such as:

- Setting secure HTTP headers.
 - Avoiding displaying stack traces or raw error messages.
 - Sanitizing user input, especially in web applications, to prevent malicious injection attacks.
-

5. Conclusion

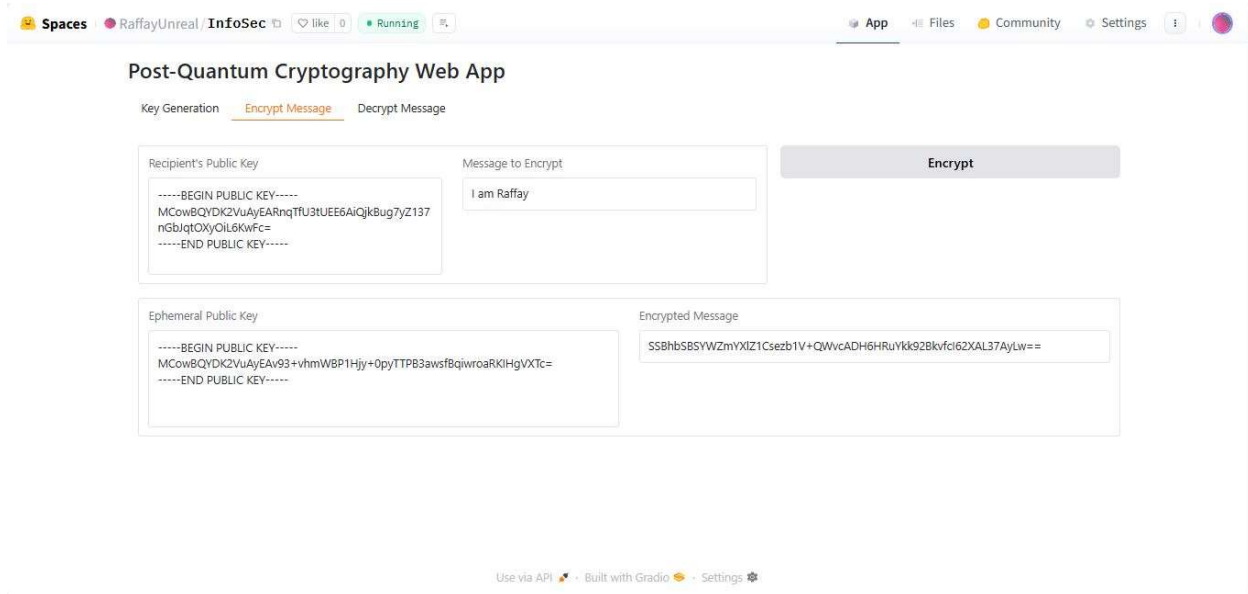
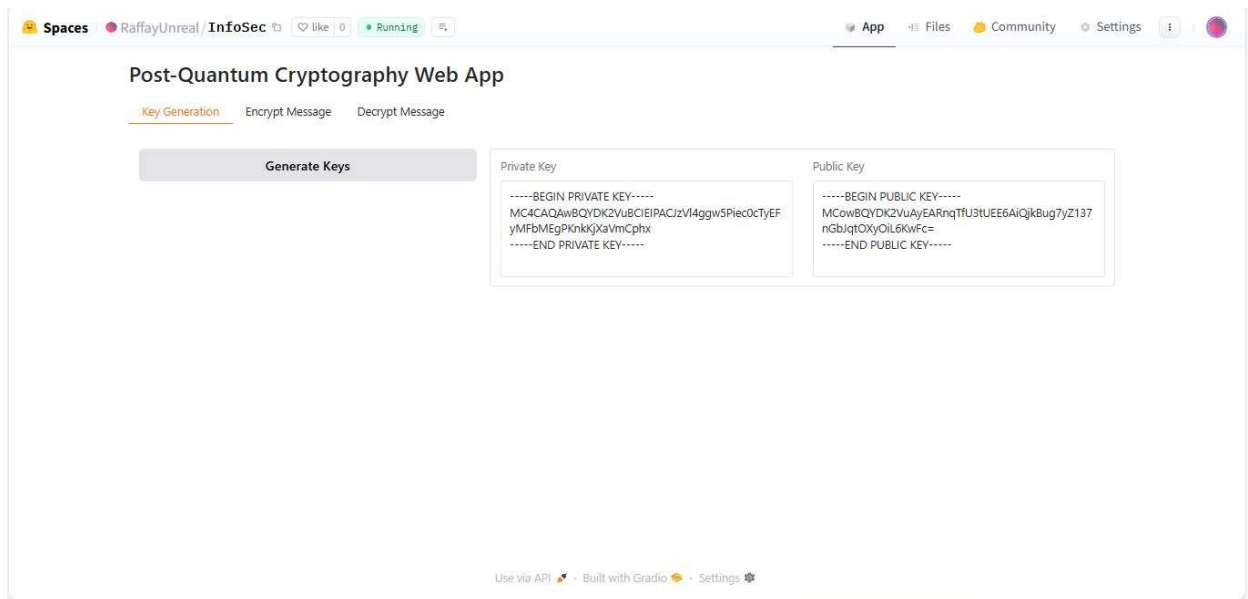
This report described the integration of a **Post-Quantum Cryptography (PQC)** algorithm using the **X25519 elliptic curve** in a web application. The application demonstrates how to generate public/private key pairs, encrypt messages using a public key, and decrypt them using the corresponding private key.

While X25519 is not strictly a PQC finalist, its use in this demonstration is motivated by its security and resistance to certain quantum threats. The system is built to be secure, efficient,

and user-friendly, offering an essential introduction to post-quantum cryptographic principles and their application in modern web systems.

Future work could involve integrating a true **post-quantum cryptography** scheme, such as **Kyber** or **Dilithium**, which are more robust against quantum attacks.

Screenshots of the App:



Spaces

RaffayUnreal

InfoSec

like 0

Running

App

Files

Community

Settings

Post-Quantum Cryptography Web App

Key Generation

Encrypt Message

Decrypt Message

Your Private Key

-----BEGIN PRIVATE KEY-----
MC4CAQAwBQYDK2VubCIEIPACIzVI4g
gw5Piec0cTYeFyMFbMEgPKnoXaVm
Cphx
-----END PRIVATE KEY-----

Sender's Ephemeral Public Key

-----BEGIN PUBLIC KEY-----
MCowBQYDK2VUAyEAy93+vhmWBP1
Hjy+0pyTTPB3awsfBqiwroaRKIHgVXTc
=
-----END PUBLIC KEY-----

Encrypted Message

SSBhbSBsYWZmYXZlCsezbt1V+QWVc
ADH6HRuYkk92Bkvc62XAL37AyLw=
=

Decrypt

Decrypted Message

I am Raffay

Use via API

Built with Gradio

Settings